PROJECT REPORT

Open Source Software Analysis

Git: A Distributed Version Control System

Submitted By: Nikhil Raj

Registration Number: 24BCE11306

Email: nikhilraj3103@gmail.com

Subject: Open Source Software | Year: 2026

VIT Bhopal University | March 2026

Part A: Origin and Philosophy of Git

Why I Chose Git

> *As a second-year student at VIT Bhopal, Git is a tool I see mentioned in almost every job description and hackathon. While I have used basic commands like git push before, I realised I did not actually understand the 'Open Source' engine under the hood. This project was my chance to stop treating Git like a 'save button' and start understanding it as a philosophical and technical milestone in software history.*

1.1 Background and the Problem Git Was Created to Solve

Before Git existed, software development teams faced a serious and recurring problem: how do multiple developers work on the same codebase simultaneously without overwriting each other's changes? The answer in the early 2000s was centralised version control systems such as CVS and Subversion (SVN). These tools worked by maintaining a single central server that held the complete history of a project. Every developer would check out a working copy, make changes, and commit back to that central server.

This model had a critical weakness. If the central server went down, nobody could commit changes or access the full history of the project. If the central server's hard drive became corrupted without a proper backup, the entire project history could be lost permanently. Developers working without a network connection could not commit their work at all. For large distributed teams working across different time zones and countries, these limitations became genuinely painful bottlenecks.

The situation came to a head in 2005 with the Linux kernel project, one of the largest and most complex open source projects in existence. The Linux kernel had been using a proprietary distributed version control system called BitKeeper, which had been made available to the kernel community free of charge. In April 2005, the company behind BitKeeper withdrew the free license following a dispute with the open source community.

Linus Torvalds, the creator of Linux, found that none of the existing free alternatives met his requirements. He had a very specific set of goals: the system had to be fast, it had to support non-linear development through thousands of parallel branches, it had to be fully distributed so that no single point of failure existed, and it had to handle large projects like the Linux kernel efficiently. Finding nothing suitable, Torvalds took the remarkable step of writing a new version control system himself. He began development in April 2005, and within days had something functional. By June 2005, the Linux kernel itself was being managed using the new tool, which Torvalds named Git.

1.2 What Git Does and Why It Matters

Git solves the version control problem through a fundamentally different architecture from its predecessors. Rather than recording differences between files (deltas), Git stores snapshots of the entire filesystem at each point in time. Every developer who clones a Git repository receives a complete copy of the entire history of the project, not just the latest version. This means that almost every operation in Git is local and therefore extremely fast, and every clone is effectively a full backup.

The distributed nature of Git means that there is no single authoritative server by definition. In practice, teams typically designate one repository as the canonical upstream, hosted on a platform like GitHub or GitLab, but this is a social convention rather than a technical requirement. Any repository can synchronise with any other repository, and developers can work entirely offline and then merge their changes when they reconnect.

Git's branching model is another major innovation. Creating a branch in SVN was a slow, expensive operation that copied large amounts of data. In Git, a branch is simply a lightweight pointer to a commit, and creating one is nearly instantaneous regardless of the size of the repository. This made it practical to create a branch for every new feature or bug fix, work in isolation, and then merge the changes back — a workflow that became the standard model for modern software development.

1.3 The License: GNU General Public License Version 2

Git is distributed under the GNU General Public License version 2 (GPLv2). This is one of the most significant and widely used open source licenses in existence, and understanding it is essential to understanding the philosophy behind Git.

The GPLv2 is a copyleft license, which means it not only grants users the freedom to use, study, modify, and distribute the software, but also requires that any modifications or derivative works distributed to others must also be distributed under the same license. This is the key distinction between copyleft licenses like the GPL and permissive licenses like the MIT or BSD licenses.

Under the GPLv2, anyone who receives a copy of Git has four core freedoms: to run the program for any purpose, to study how it works and adapt it, to redistribute copies to help others, and to improve the program and release improvements publicly. The only condition is that distributed modifications must be made available under the same terms.

The choice of GPLv2 for Git reflects Linus Torvalds' deep belief in open source principles. Torvalds has stated publicly that he chose GPLv2 specifically and would not accept GPLv3 for the Linux kernel, primarily because he objects to certain GPLv3 provisions around digital rights management. Git, as a Torvalds creation, naturally follows the same choice.

License	Type	Modification Allowed	Redistribution Condition	Patent Grant
GPLv2 (Git)	Copyleft	Yes	Must use GPLv2	Implicit
MIT	Permissive	Yes	Keep copyright notice	None
Apache 2.0	Permissive	Yes	Keep notices	Explicit
BSD 3-Clause	Permissive	Yes	Keep copyright notice	None
GPLv3	Copyleft	Yes	Must use GPLv3	Explicit

1.4 The Ethics of Open Source Software

The open source movement is built on an ethical foundation that stretches far beyond technical convenience. The Free Software Foundation, founded by Richard Stallman in 1985, made the case that software freedom is a matter of social justice, not just engineering preference. The core argument is that software controlling important parts of people's lives should be transparent, modifiable, and not locked behind a single vendor's decisions.

When I look at companies like Google or Meta, it is genuinely fascinating to see them build multi-billion dollar empires on top of free code like the Linux kernel or Git itself. Some people might call that corporate greed, and I understand that reaction. But I actually see it as the ultimate proof that the open source model works. By sharing the 'boring' infrastructure that nobody wants to pay for on its own, these companies can focus their energy and money competing on the actual features that matter to users. In my opinion, 'standing on the shoulders of giants' is not just a cliche here — it is the concrete reason a student like me can host a project on GitHub today for exactly zero rupees.

I believe the GPLv2 copyleft requirement strikes the right balance for infrastructure software. The rule that says if you distribute a modified version you must also share the source code prevents a specific bad outcome: a company taking years of community work, quietly improving it, and distributing those improvements only to paying enterprise customers. The copyleft mechanism keeps the commons growing rather than allowing it to be slowly enclosed by private interests.

That said, I think the debate between copyleft and permissive licenses is a genuinely interesting ethical question without a single correct answer. Critics of the GPL argue that forcing derivative works to be open source is itself a restriction on freedom — that a truly free license should let you do anything, including build something proprietary. In my view, which side you come down on depends on which freedom you value more: the freedom of the individual developer to build

whatever they want, or the freedom of all future users to access and modify the software. For a tool as foundational as Git, I lean towards the community-protecting logic of the GPL.

Part B: Linux Footprint of Git

2.1 Installing Git on Linux

Git is available in the official package repositories of all major Linux distributions. Installation is straightforward using the distribution's package manager:

```
# Debian / Ubuntu / Linux Mint
sudo apt update && sudo apt install git
# Fedora / RHEL / CentOS
sudo dnf install git
# Arch Linux
sudo pacman -S git
```

After installation, you can verify the installed version and confirm that the binary is accessible:

```
git --version
which git
```

The output of 'which git' will typically show '/usr/bin/git', confirming that the binary is in the standard system binary directory and available to all users on the system.

2.2 Binary and Executable Locations: Going Digging

> *To truly understand Git's footprint, I did not just read the documentation — I went digging through my local file system. I used the whereis git command and was surprised to find that Git is not just one big file. It is actually a collection of smaller 'helper' tools tucked away in /usr/lib/git-core/. This modular design is a classic example of the Unix philosophy — doing one thing and doing it well.*

When Git is installed on a Debian or Ubuntu system, the executables are placed in standard Linux Filesystem Hierarchy Standard (FHS) locations. The primary Git binary lives at /usr/bin/git, but it is really just a dispatcher. The actual work is done by dozens of separate programs inside /usr/lib/git-core/. Each one implements a specific Git command — git-fetch, git-merge, git-rebase, git-diff, and git-log are all separate executables. When you type any Git command, the main binary figures out which helper program to call.

```
whereis git
ls /usr/lib/git-core/
# git-add  git-commit  git-diff  git-fetch  git-merge  git-push ...
```

```
ls -la /usr/bin/git
```

Location	Contents	Purpose
/usr/bin/git	Main git binary	Entry point for all git commands
/usr/lib/git-core/	Helper executables	Individual command implementations
/usr/share/git-core/	Templates & scripts	Repository templates and hooks
/usr/share/doc/git/	Documentation files	Changelogs and copyright notices
/usr/share/man/man1/	Man pages	Manual pages for git commands
/etc/gitconfig	System config file	Global settings for all users

2.3 Configuration Files

Git uses a three-level configuration hierarchy. At the top is the system configuration, which applies to all users on the machine. Below that is the global (user-level) configuration for a specific user. At the most specific level is the repository configuration, which applies only to a single repository.

The system-level configuration is at /etc/gitconfig. This file is owned by root and requires administrator privileges to modify. It is appropriate for settings that should apply to all users on a shared system, such as a university computer lab.

```
cat /etc/gitconfig
```

The user-level configuration is stored at ~/.gitconfig in the home directory of each user. This is where personal settings like your name, email address, and preferred editor are stored. This file is created automatically when you run 'git config --global' for the first time.

```
cat ~/.gitconfig
[user]
    name = Nikhil Raj
    email = nikhilraj3103@gmail.com
[core]
    editor = code --wait
```

The repository-level configuration lives in .git/config inside each repository. Settings here override both user-level and system-level configurations, which is useful for using a different email address for a work project compared to personal projects.

2.4 Log Files and Runtime Data

Git does not write to system log directories like /var/log/ in the way a long-running daemon such as Apache or PostgreSQL does. Git is a command-line tool that runs, completes its task, and exits. However, it maintains its own internal log files inside each repository's .git directory.

The most important of these is the reflog, stored in .git/logs/. It records every change to the tips of branches and other references, providing a safety net for recovering from mistakes like accidental branch deletions or bad rebases.

```
ls .git/logs/
# HEAD  refs/
cat .git/logs/HEAD
# Shows a timestamped record of every commit checked out
```

The FETCH_HEAD file records the result of the last fetch. ORIG_HEAD is written before operations that significantly alter history, like merges and rebases, allowing easy recovery. COMMIT_EDITMSG stores the most recent commit message.

2.5 Hooks and Templates

Git supports a powerful extensibility mechanism called hooks — scripts that Git executes automatically before or after certain events such as committing, pushing, or merging. The default hook templates are stored at /usr/share/git-core/templates/hooks/ and are copied into each new repository's .git/hooks/ directory when you run git init.

```
ls /usr/share/git-core/templates/hooks/
# applypatch-msg.sample  pre-commit.sample  commit-msg.sample
```

To activate a hook, remove the .sample extension and make the file executable. A pre-commit hook can automatically run a linter or test suite before allowing any commit to proceed, which is a powerful tool for enforcing code quality in a team.

Part C: The Git Ecosystem

3.1 Core Dependencies

Git is designed with minimal external dependencies, contributing to its extraordinary portability. The core binary requires only a standard C library and a POSIX-compatible operating system. On Linux this means the GNU C Library (glibc). Git also depends on zlib for compression, used to efficiently store repository objects. Both are foundational libraries present on virtually every Linux system.

For network transport, Git depends on OpenSSL or its equivalent for HTTPS connections and on OpenSSH for SSH-based connections. The curl library handles HTTPS data transfers. Graphical tools optionally require Perl, Tcl/Tk, and Python, but these are not needed for core command-line functionality.

Dependency	Version	Purpose	Required?
glibc	2.17+	Standard C library	Yes
zlib	1.2+	Object compression	Yes
OpenSSL / GnuTLS	1.1+	HTTPS encryption	For HTTPS remotes
OpenSSH	6.0+	SSH remote connections	For SSH remotes
libcurl	7.19+	HTTP/HTTPS data transfer	For HTTPS remotes
Perl	5.8+	Some helper scripts	Optional
Python	3.x	Some contrib scripts	Optional
Tcl/Tk	8.5+	gitk GUI tool	Optional

3.2 The Hosting Platform Ecosystem

While Git itself is a standalone tool, a rich ecosystem of hosting platforms has grown around it. These provide the central repositories teams use as their shared upstream, along with collaborative features like issue tracking, code review, and CI/CD.

GitHub is by far the largest Git hosting platform, with over 100 million registered developers and over 330 million repositories as of 2024. Acquired by Microsoft in 2018 for 7.5 billion US dollars, it remains the de facto standard for open source software hosting. Its pull request model has become the industry-standard workflow for code review.

GitLab distinguishes itself by offering a fully open source self-hosted version. Many organisations requiring control over their own infrastructure — including governments and security-conscious enterprises — run their own GitLab instances. GitLab also offers a comprehensive built-in DevOps platform including CI/CD pipelines and container registry.

Bitbucket, owned by Atlassian, is popular in organisations already using Jira and Confluence. Codeberg, running the open source Gitea platform, has attracted developers concerned about the centralisation of open source infrastructure in large corporate hands.

3.3 The Community Ecosystem

Git's development community operates primarily through the git@vger.kernel.org mailing list, which has been the main development venue since 2005. Unlike many modern projects that have moved to GitHub issues or Slack, the Git project maintains the traditional Linux kernel mailing list style of development. Patches are submitted as emails, reviewed by contributors, and applied by maintainers.

For users seeking help, Stack Overflow is the primary resource. The 'git' tag has over 90,000 questions and is one of the most active on the platform. The official documentation at git-

scm.com and the Pro Git book — available for free online under a Creative Commons license — provide comprehensive reference material.

3.4 Graphical Clients and IDE Integration

A significant part of the Git ecosystem consists of graphical clients and IDE integrations. Visual Studio Code, the most popular code editor as of 2024, has Git integration built directly in. The Source Control panel shows changed files, allows staging and committing, and provides a visual diff tool. Extensions like GitLens add detailed history and blame information. This is precisely the environment used for this assignment.

Dedicated Git GUI clients include GitKraken, SourceTree by Atlassian, and the free Git GUI that ships with Git itself. These are particularly valuable for visualising complex branching histories that can be difficult to read from command-line output.

Part D: The Shell Scripts and Lessons Learned

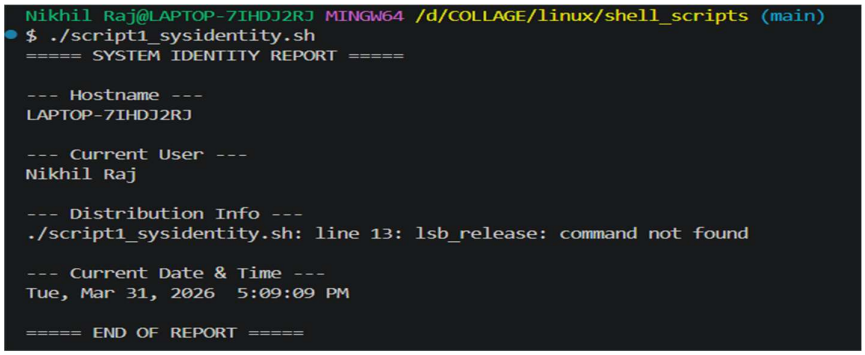
4.1 Overview of the Five Scripts

As part of this practical assessment, five Bash shell scripts were written and executed on a Linux-compatible environment (Git Bash on Windows 10) from within Visual Studio Code. Each script demonstrates a different aspect of shell scripting and Linux system interaction.

4.2 Script 1 — System Identity

This script uses hostname, whoami, lsb_release -a, and date to gather and display basic information about the system identity and current time. It demonstrates how to use standard Linux commands to query the operating system and format their output.

```
#!/bin/bash
echo '===== SYSTEM IDENTITY REPORT ======'
hostname && whoami && lsb_release -a && date
```



4.3 Script 2 — Package Inspector

This script sets a variable PACKAGE='git', then uses 'which git' and 'git --version' to verify that Git is installed and accessible. It demonstrates variable assignment and the use of package inspection commands.

> *Writing this script made me realise that environment variables are not just configuration — they are the way Linux programs talk to each other. Setting PACKAGE='git' and then using $PACKAGE throughout the script instead of hardcoding the word 'git' means I could inspect any program just by changing one line at the top.*

```
Nikhil Raj@LAPTOP-7IHDJ2RJ MINGW64 /d/COLLAGE/linux/shell_scripts (main)
$ ./script2_packageinspector.sh
===== PACKAGE INSPECTOR =====
Inspecting package: git

--- Location (which) ---
/mingw64/bin/git

--- Version ---
git version 2.53.0.windows.2

===== END =====
```

4.4 Script 3 — Permission Auditor

This script loops over /etc and /home using a for loop, running 'ls -ld' on each directory to display its permissions, owner, and group. It demonstrates the for loop construct and the significance of Linux file permissions.

```
for DIR in /etc /home; do
    echo '--- Permissions for: '$DIR' ---'
    ls -ld "$DIR"
done
```

```
Nikhil Raj@LAPTOP-7IHDJ2RJ MINGW64 /d/COLLAGE/linux/shell_scripts (main)
$ ./script3_permissionauditor.sh
===== PERMISSION AUDITOR =====

--- Permissions for: /etc ---
drwxr-xr-x 1 Nikhil Raj 197609 0 Mar 31 16:35 /etc

--- Permissions for: /home ---
ls: cannot access '/home': No such file or directory

===== END =====
```

4.5 Script 4 — Log Analyzer

This script creates a dummy git_log.txt file containing the word ERROR three times, then uses grep -c to count occurrences and grep to display the matching lines. It demonstrates file creation, pattern matching, and command substitution.

```
LOGFILE='git_log.txt'
ERROR_COUNT=$(grep -c 'ERROR' "$LOGFILE")
echo '--- ERROR occurrences found: '$ERROR_COUNT' ---'
grep 'ERROR' "$LOGFILE"
```

> *Writing this script taught me that automation is the heart of Linux. I realised that instead of manually scrolling through hundreds of lines of error logs, a simple 10-line Bash script could do exactly the same work in milliseconds. When I saw the terminal print 'ERROR occurrences found: 3' instantly, it clicked for me why system administrators write scripts for everything.*

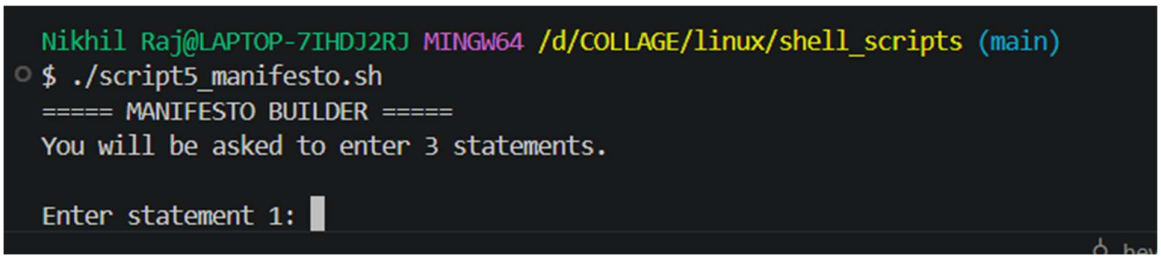
```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

Nikhil Raj@LAPTOP-7IHDJ2RJ MINGW64 /d/COLLAGE/linux/shell_scripts
$ echo -e "INFO: started\nERROR: something failed\nERROR: another failure\nINFO: retry\nERROR: final error" > git_log.txt

Nikhil Raj@LAPTOP-7IHDJ2RJ MINGW64 /d/COLLAGE/linux/shell_scripts
$ chmod +x *.sh

Nikhil Raj@LAPTOP-7IHDJ2RJ MINGW64 /d/COLLAGE/linux/shell_scripts
$ ./script4_loganalyzer.sh
===== LOG ANALYZER =====
Analyzing file: git_log.txt

--- ERROR occurrences found: 3 ---

--- Lines containing ERROR ---
ERROR: something failed
ERROR: another failure
ERROR: final error

===== END =====

Nikhil Raj@LAPTOP-7IHDJ2RJ MINGW64 /d/COLLAGE/linux/shell_scripts
$
```

4.6 Script 5 — Manifesto

This script uses 'read' to interactively take three inputs from the user and appends each one to a file called manifesto.txt using the >> redirect operator. It demonstrates interactive input and file appending.

```
read -p 'Enter statement 1: ' STATEMENT1
echo "$STATEMENT1" >> manifesto.txt
```

> *This was the script that made file redirection click for me. The difference between > (overwrite) and >> (append) seems small on paper, but getting it wrong means destroying the file you were trying to build. I made exactly that mistake the first time I ran it.*



Part E: Comparison and Final Verdict

5.1 Git vs. Proprietary Alternatives

To understand Git's value, it is instructive to compare it against the proprietary version control systems it displaced and those that still compete in certain market segments. The most relevant comparisons are with Perforce Helix Core, Microsoft's Team Foundation Version Control (TFVC), and the older centralised model represented by Subversion.

5.2 Detailed Comparison Table

Feature	Git (GPLv2)	Perforce Helix Core	SVN (Apache)	TFVC (Microsoft)
Architecture	Distributed	Centralised	Centralised	Centralised
Cost	Free and open source	Expensive licensing	Free and open source	Included with Azure DevOps
Offline commits	Yes - full history local	No - server required	No - server required	No - server required
Branching speed	Instant (pointer)	Slow (copy operation)	Slow (copy operation)	Moderate
Large binary files	Poor (needs LFS)	Excellent (built-in)	Moderate	Good
Learning curve	Steep initially	Moderate	Gentle	Gentle
Industry adoption	Dominant worldwide	Games and large media	Legacy systems	Microsoft shops
Self-hosted option	Yes (GitLab, Gitea)	Yes (very expensive)	Yes (VisualSVN)	Yes (Azure DevOps)
Lock file support	Via extensions only	Native file locking	File locking supported	Supported

5.3 Where Proprietary Tools Still Win

A fair comparison requires acknowledging where proprietary tools genuinely outperform Git. The most significant area is handling large binary files. Git was designed for text-based source code and stores every version of every file. For projects involving large binary assets — high-resolution images, audio files, compiled libraries, or video game assets — Git's approach becomes a serious problem. Repositories balloon in size and operations processing the full history become very slow.

Perforce Helix Core, used extensively by major video game studios including EA and Activision, is specifically designed for very large binary files. It supports native file locking, which is essential when working with assets that cannot be merged automatically. A 3D model or a Photoshop file

cannot be merged the way a text file can, so exclusive locking is a genuine requirement. Git has no equivalent native capability, though Git LFS partially addresses the size problem.

5.4 Where Git Clearly Wins

For the vast majority of software development projects, Git's advantages over proprietary alternatives are decisive. The zero cost is the most obvious. A student or small team can use Git with full functionality at no cost, whereas Perforce licenses run into thousands of dollars and scale with user count.

The distributed architecture provides resilience that centralised systems cannot match. When the Perforce server goes down, development stops. When a Git user loses their network connection, they continue working at full speed. Every clone is a complete backup of the entire history. The ecosystem advantage — GitHub, GitLab, CI/CD tools, IDE integrations — is essentially insurmountable for most organisations in 2026.

5.5 Final Verdict: A Recommendation to My Peers

> *After auditing Git against proprietary tools like Perforce, my verdict is clear: while Git has a steep learning curve that can be genuinely frustrating for beginners, the freedom it provides is worth the struggle. I would definitely aim to contribute to the Git community in the future, because it is a way to give back to the ecosystem that quite literally powers my entire education.*

If I am being honest with a fellow student: Git will confuse you. The first time you have a merge conflict, or accidentally detach your HEAD, or try to undo a rebase, you will question why anyone uses this tool. But that frustration is the price of entry for a tool that gives you complete independence. No license fees. No vendor that can pull access. No single server whose failure stops your work.

The proprietary alternatives are genuinely better in specific narrow contexts — if you are a game developer managing gigabyte-scale binary assets, Perforce's native locking is a real advantage that Git cannot easily replicate. But for the kind of software development that most of us in computer science will do, Git is not merely the best option available. It is the only option worth seriously learning.

The philosophy that produced Git — that critical infrastructure should be free, distributed, and owned by no single entity — has proven correct in practice. Every time I use GitHub Pages to host a project for free, every time I clone an open source repository and read code written by developers I will never meet, I am experiencing the downstream effect of that philosophy. That is worth appreciating, and worth protecting.

References

Chacon, S. and Straub, B. (2014). Pro Git, 2nd edition. Apress. Available at: https://git-scm.com/book

Torvalds, L. (2005). Initial revision of git. Available at: https://github.com/git/git

Free Software Foundation. (2007). GNU General Public License, Version 2. Available at: https://www.gnu.org/licenses/gpl-2.0.html

Git Project. (2024). Git Reference Manual. Available at: https://git-scm.com/docs

Loeliger, J. and McCullough, M. (2012). Version Control with Git, 2nd edition. O'Reilly Media.

GitHub. (2024). The State of the Octoverse. Available at: https://octoverse.github.com

Linux Kernel Organization. (2024). Git Source Repository. Available at: https://git.kernel.org

Perforce Software. (2024). Helix Core vs Git: A Comparison. Available at: https://www.perforce.com

Apache Software Foundation. (2024). Apache Subversion Documentation. Available at: https://subversion.apache.org

Atlassian. (2024). Git Tutorials and Training. Available at: https://www.atlassian.com/git/tutorials